



Л Е К Ц И Я 0 5

Протоколы валидации, воспроизводимость и утечки данных

Курс «Машинное обучение и безопасность систем»
Магистратура НИУ ВШЭ (МИЭМ) · ОП «Информационная безопасность и технологии ИИ» · 1 курс

Автор: Силаев Ю. В. · Длительность изучения: ~65 минут · 54 слайда



О чём эта лекция за минуту

2 / 54

Часть 1 · Введение

Главный тезис, который мы будем разворачивать все 65 минут

Качество ML-модели определяется не только моделью, **но и протоколом эксперимента**. Неправильное разбиение данных, утечки признаков, подгонка под тестовую выборку и невозпроизводимый запуск могут полностью обесценить даже технически сильную модель – особенно в задачах информационной безопасности.

Что мы пройдем

Разбиение данных

Train / validation / test, holdout, K-fold, stratified, time-based, group split. Когда какая схема корректна, а когда даёт иллюзию качества.

Утечки данных

Шесть концептуальных типов data leakage. Где они прячутся в ИБ-данных и почему случайное перемешивание – самая частая ошибка студентов.

Воспроизводимость

Минимальный набор сведений, без которого эксперимент нельзя повторить. Журнал запусков как обязательный артефакт работы.



Что вы будете уметь после лекции

3 / 54

Часть 1 · Введение

Семь практических навыков, которые проверяются в ДЗ и проекте

После изучения L05 вы научитесь:

- 1 Различать роли train, validation и test set и не путать их функции в эксперименте.
- 2 Выбирать схему разбиения с учётом временной структуры, групп и зависимостей между наблюдениями.
- 3 Объяснять назначение cross-validation, понимать её ограничения и не применять там, где она вводит в заблуждение.
- 4 Выявлять и классифицировать типовые источники data leakage по шести концептуальным типам.
- 5 Применять принцип «сначала разделить – потом обрабатывать» к препроцессингу (концептуально; техника – в L10).
- 6 Описывать минимальный набор требований к воспроизводимости ML-эксперимента и вести экспериментальный журнал.
- 7 Объяснять, почему нельзя настраивать модель по test set, и распознавать эту ошибку в чужих и своих работах.



Эта лекция опирается на L02-L04 и не требует продвинутых тем

Должно быть уже пройдено

L02 – Данные и EDA. Природа зависимостей в данных, понятие группы и временной структуры. Это фундамент для выбора схемы разбиения в L05.

L03 – Постановка задачи и baseline. Понятие объекта, признака, целевой переменной, концепция baseline как обязательного элемента эксперимента.

L04 – Метрики качества. Precision, recall, F1, ROC AUC, цена ошибок. В L05 эти метрики будут считаться на разных выборках – нужно понимать, что именно они измеряют.

Не понадобится в этой лекции

Линейные модели и регуляризация (идут в L06-L07).

Pipeline и ColumnTransformer (техническая реализация – в L10; здесь только концепция).

Деревья, ансамбли и нейросети (L08, L13 и далее).

Формальные доказательства несмещённости оценок и bootstrap-доверительные интервалы – за рамками курса.



План лекции

5 / 54

Часть 1 · Введение

Семь частей · 54 слайда · ~65 минут самостоятельного изучения

Часть	Название	Слайды	Время изучения
1	Введение в лекцию	1-5	~6 мин
2	Мотивация: иллюзия качества	6-9	~5 мин
3	Основное содержание	10-42	~40 мин
4	Связки с практикой ИБ	43-48	~7 мин
5	Итоги	49-51	~3 мин
6	Источники для углубления	52-53	~2 мин
7	Глоссарий	54	~2 мин



Почему «AUC = 0.99» иногда ничего не значит

Расхождение между метрикой на отчёте и поведением модели в эксплуатации

В задачах информационной безопасности встречается характерный паттерн: модель показывает **почти идеальные числа на тестовой выборке** – а на боевых данных её качество падает в разы. Причина – почти всегда не модель, а протокол эксперимента.

Сценарий	На отчёте (test)	В эксплуатации	Скрытая причина
Malware-классификатор	F1 = 0.97	F1 $\approx$ 0.60	Файлы одного семейства в train и test
Детектор фишинга	AUC = 0.99	AUC $\approx$ 0.78	Шаблоны одной кампании в обеих выборках
Сетевые аномалии	Recall = 0.95	Recall $\approx$ 0.40	Случайный split на временных логах

Общая черта всех трёх сценариев. Модель не «плохая» – она хорошо выучила то, что ей показали. Но показали ей выборку, устроенную не так, как реальный поток данных. Каждая из этих причин – отдельный тип проблемы, и каждой в лекции посвящён свой раздел.



ИБ-кейс: malware-классификатор, который «работал»

Реальный паттерн из практики SOC-команд – числа отличные, защита не работает

ИБ-КЕЙС

Классификатор PE-файлов на 12 семействах вредоносного ПО

Постановка. ~40 000 PE-файлов с метками 12 семейств. Признаки – N-граммы байт-кода, импортируемые библиотеки, энтропия секций. Случайный split 70 / 15 / 15. Модель – градиентный бустинг.

На отчёте. Макро-F1 = 0.97 на test, ROC-AUC $\approx$ 0.99. Модель развёрнута в SOC как фильтр первой линии.

В проде. Через 3 недели F1 упал до $\approx$ 0.60. На новых семействах модель почти не работает; на старых – путает полиморфные варианты.

⚠ ДИАГНОЗ

Полиморфные варианты одного семейства похожи между собой сильнее, чем разные семейства между собой. При случайном split «братья-близнецы» оказываются и в train, и в test – модель просто узнаёт уже виденные семейства, а не учится их различать.

*В литературе это известно как **family leakage**. Концептуальный язык, чтобы его описать, мы получим в Части 3, блок 3.*



Карта проблем, которые мы будем разбирать в основном содержании лекции

Когда числа на бумаге расходятся с поведением в проде, причина почти всегда сводится к одному из трёх источников. Каждому посвящён отдельный раздел в Части 3.

Неправильное разбиение

1

Случайный split там, где есть время, группы или зависимости между объектами. Train и test перестают быть независимыми, и оценка качества завышается.

Пример: события одного пользователя попадают и в train, и в test.

→ разбирается в Части 3, блок 2

Утечка данных (data leakage)

2

Информация, недоступная модели на момент реального прогноза, всё-таки просочилась в обучение. Шесть концептуальных типов – от утечки через признаки до утечки через дубликаты.

Пример: признак, рассчитанный после реакции SOC-аналитика.

→ разбирается в Части 3, блок 3

Подгонка под test

3

Множественные сравнения моделей по test или подбор гиперпараметров по нему. Каждый «взгляд» на test и улучшение после него – статистическая, но настоящая форма утечки.

Пример: «я обучил 30 моделей и взял лучшую по test».

→ разбирается в Части 3, блок 5

Три источника часто действуют вместе. В кейсе со слайда 7 работают первый источник (group-связь по семейству) и третий (повторные взгляды на test).



Ключевое утверждение

“

Даже сильная модель не имеет доказанного качества, если эксперимент построен с утечками, некорректным разбиением данных или невозпроизводимым протоколом.

— *ключевой тезис*

Из этого тезиса вытекают три рабочих принципа:

- 1. Сначала фиксируем протокол эксперимента, потом обучаем модель. Не наоборот.**
- 2. Любая оценка качества – это пара «число + протокол».** Число без протокола не имеет смысла.
- 3. Результат, который нельзя воспроизвести, не считается результатом – это эпизод, а не знание.**



Ч А С Т Ь 3 · Б Л О К 1

Зачем нам три выборки

Train, validation и test – три роли, которые нельзя путать

В этом блоке (слайды 10-14):

- 11 Три роли выборок – формальные определения
- 12 Почему именно три, а не две
- 13 Частая ошибка: «test и validation – одно и то же»
- 14 Мини-резюме блока



Ключевые определения

ОПРЕДЕЛЕНИЕ

TRAIN SET

Обучающая выборка

Данные, на которых модель подбирает свои параметры – веса, коэффициенты, структуру деревьев.

Используется внутри метода обучения. Метрики на ней показывают, насколько модель запомнила обучающие примеры.

VALIDATION SET

Валидационная

Данные, на которых выбирается модель: гиперпараметры, тип алгоритма, момент остановки обучения.

Используется снаружи метода обучения. Можно смотреть много раз – это и есть её назначение.

TEST SET

Тестовая выборка

Финальная независимая оценка качества уже выбранной модели на данных, которых не касалась настройка.

Смотрится один раз. Используется для отчёта о качестве, не для улучшения модели.

Ключевое различие – кто смотрит на выборку. На train – сам алгоритм. На validation – инженер при выборе. На test – один раз, для отчёта. Подмена одной роли другой – главный источник иллюзии качества.



Логика трёх выборок и где здесь возникает зависимость, которой мы не хотим

Цепочка использования выборок

1

Train

Обучаем модель – алгоритм оптимизирует параметры под train.

2

Validation

Выбираем модель – сравниваем гиперпараметры, архитектуры, признаки.

3

Test

Один раз измеряем качество выбранной модели – для честного отчёта.

Почему двух выборок мало

Допустим, у нас только train и test. Мы обучили модель, посмотрели на test, увидели низкое качество – поменяли гиперпараметры – снова посмотрели на test – и так несколько раз.

Что произошло. Test перестал быть независимым: **мы выбрали по нему модель**. Любое улучшение метрики после взгляда на test – это форма подгонки под него. Финальное число – уже не оценка обобщения, а максимум, которого мы достигли в подгонке.

Решение. Развести роли: validation – для выбора (можно смотреть много раз), test – для финального отчёта (смотрим один раз).

Связь с L04. Precision, recall, F1, ROC AUC и любые другие метрики считаются **на всех трёх выборках** – и означают разное. Метрика на train $\neq$ метрика на validation $\neq$ метрика на test.



Самое распространённое заблуждение по теме блока 3.1

⚠ ЧАСТАЯ ОШИБКА

«У меня уже есть *train* и *test*. Зачем мне ещё какой-то *validation*? Я обучу модель на *train*, посмотрю качество на *test* – и если плохо, поменяю параметры. **Test и validation – это просто разные названия одной и той же выборки**».

✗ ПОЧЕМУ ЭТО НЕВЕРНО

Подмена ролей создаёт скрытую утечку. Как только вы посмотрели на «test» и поменяли гиперпараметры – этот test перестал быть независимым. Вы **выбрали модель по нему**, и финальная цифра показывает уже не обобщение, а максимум подгонки.

Незаметность – главная опасность. Цифра растёт, кажется, что модель улучшается. На самом деле улучшается только её соответствие именно этой выборке – в проде такого качества не будет.

Правильно так. Validation – для выбора (смотрим много раз). Test – для отчёта (смотрим один раз, и больше модель не трогаем).

→ детальное обсуждение работы с test set – в блоке 3.5 (слайды 37-40)



Мини-резюме блока 3.1: train / validation / test

Главные тейкэвеи

- 1** **Train, validation и test – три разные роли,**
а не три случайно нарезанных куска одного датасета. Подмена одной другой ломает эксперимент.
- 2** **Test – это финальная независимая оценка.**
На неё смотрят один раз, и после этого модель не меняется. Подбор гиперпараметров по test – это leakage.
- 3** **Validation – для выбора, train – для обучения.**
На validation можно смотреть много раз, для этого она и существует. На train смотрит только сам алгоритм оптимизации.
- 4** **Метрика без указания выборки бессмысленна.**
 $F1 = 0.92$ – на чём? Train, validation или test? Это три очень разных числа.

Что дальше в Части 3

Блок 2 15-22

Схемы разбиения

Holdout, K-fold, stratified, time-based, group split.

Блок 3 23-32

Типология leakage

Шесть концептуальных типов утечек. Сердцевина лекции.

Блок 4 33-36

«Split first»

Принцип против leakage через preprocessing.

Блок 5 37-40

Работа с test

Правила обращения с финальной выборкой.

Блок 6 41-42

Воспроизводимость

Минимум, без которого эксперимент нельзя повторить.



ЧАСТЬ 3 · БЛОК 2

Как именно разбивать данные

От простой схемы к специализированным – пять подходов и nested validation

В этом блоке (слайды 15-22):

- 16 Holdout – простейшая схема
- 17 K-fold cross-validation
- 18 Stratified split
- 19 Time-based split (логи, временные данные)
- 20 Group split (связанные объекты)
- 21 Nested validation – на уровне идеи
- 22 Мини-резюме блока: карта схем



Один раз разрезали данные, на каждой части – своя роль

Схема разбиения 70 / 15 / 15

TRAIN · 70%

VAL · 15%

TEST · 15%

Один датасет – три непересекающихся куска. Каждый объект попадает ровно в одну часть.

✓ Когда holdout достаточно

- Данных много (десятки тысяч объектов и больше).
- Объекты **независимы** друг от друга – нет групп, нет времени.
- Достаточно одной оценки качества – не нужны доверительные интервалы.

⚠ Когда holdout вводит в заблуждение

- Данных мало – одна оценка качества слишком шумная.
- Есть **временная структура** (логи, события) → нужен time-based split (слайд 19).
- Есть **группы** (пользователи, семейства, кампании) → group split (слайд 20).

Типичные пропорции. 70 / 15 / 15 или 60 / 20 / 20 – для умеренных датасетов. На больших датасетах (миллионы записей) test может быть 5-10%. Главное – чтобы test был достаточно большим для надёжной оценки, а train – для обучения. **Перемешивание данных перед split обязательно** – если только это не временные данные.



Несколько разбиений вместо одного – устойчивая оценка качества

K = 5 fold: пять разбиений одного и того же датасета

Fold 1	VAL	TRAIN	TRAIN	TRAIN	TRAIN
Fold 2	TRAIN	VAL	TRAIN	TRAIN	TRAIN
Fold 3	TRAIN	TRAIN	VAL	TRAIN	TRAIN
Fold 4	TRAIN	TRAIN	TRAIN	VAL	TRAIN
Fold 5	TRAIN	TRAIN	TRAIN	TRAIN	VAL

Зачем нужна

Одна holdout-оценка может оказаться удачной или неудачной случайно – особенно на малых данных. **K-fold усредняет K оценок и даёт более устойчивую цифру.**

Что считать «качеством»

Среднее по фолдам $\pm$ стандартное отклонение – это и есть оценка обобщения. Test set всё равно **должен быть отдельным** – K-fold его не заменяет.

Каждая часть по очереди становится validation; модель обучается 5 раз. Итоговая оценка – среднее по фолдам.

⚠ ОГРАНИЧЕНИЯ

K-fold не спасает от утечек через группы или время. Случайное разбиение на K частей – то же самое перемешивание, что и в holdout. Если данные зависимы, нужны time-based или group варианты K-fold (слайды 19-20). И вторая цена – обучать модель K раз.

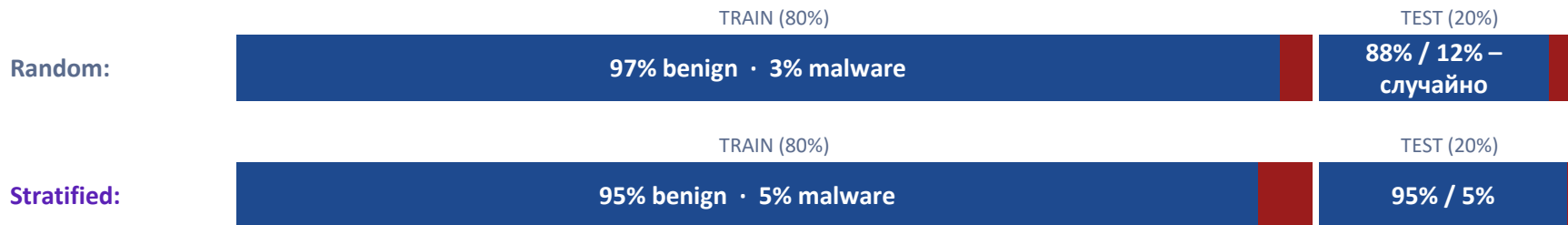


Сохранение пропорции классов между train, validation и test

ОПРЕДЕЛЕНИЕ

Stratified split – это разбиение, при котором **пропорция классов в каждой выборке (train, validation, test) совпадает с пропорцией классов во всём датасете**. В K-fold-варианте – пропорция сохраняется в каждом fold.

Пример: 95% benign, 5% malware – что получится при разбиении 80 / 20



Почему это важно в ИБ. Классы в задачах безопасности почти всегда несбалансированы – атак на порядки меньше, чем обычного трафика. При случайном split на маленьком test может оказаться 12% атак вместо 5% – и precision / recall (из L04) сместятся непредсказуемо. Stratified гарантирует, что метрики на test измеряют именно то качество, которое будет в проде.



Для логов и временных данных: только прошлое – в train, только будущее – в test

Принцип: разрез по времени, без перемешивания



✓ Почему так правильно

В проде модель всегда предсказывает **будущее по прошлому**. Test из будущего имитирует именно эту ситуацию.

Качество на «будущем» test честно показывает, как модель поведёт себя на новых, ещё не виденных событиях.

✗ Что ломает случайный split

При перемешивании события из будущего попадают в train. Модель «подсматривает» будущее – **утечка через время** (тип 5, слайд 29).

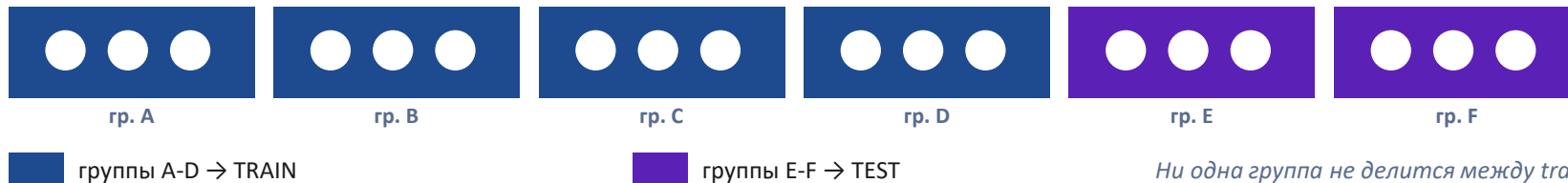
Метрика на test взлетает, но в проде будущего ещё нет – и качество рушится.

CIC-IDS-2018. Данные собраны по дням; атаки распределены по конкретным датам. Случайный split разрушает временную причинность. Правильно – обучаться на ранних днях, проверяться на поздних. → *детальный разбор time-based split для лог-данных – в L12.*



Связанные объекты должны целиком уходить в одну выборку

Принцип: группа неделима – все её объекты в одной части



Что такое «группа» в ИБ

Один пользователь, одно устройство, одна сессия, семейство файлов, кампания атаки, один IP-источник – всё, что порождает **похожие объекты**.

✗ Кейс со слайда 7 – теперь понятен

Malware-классификатор провалился именно из-за отсутствия group split: **файлы одного семейства попали и в train, и в test**.
Group split по семейству – единственная честная оценка.

CIC-IDS-2018: одни и те же IP-источники между train и test – это нарушение group split. → *применение group split для аномалий – в L11.*



Идея двух вложенных циклов: один выбирает модель, другой честно её оценивает

Два контура

ВНЕШНИЙ ЦИКЛ – оценка обобщения

делит данные на «оценочные» фолды; каждый по очереди – финальная проверка

ВНУТРЕННИЙ ЦИКЛ – подбор гиперпараметров

на каждом шаге внешнего цикла отдельно подбирает гиперпараметры – **только на обучающей части**, не касаясь оценочного фолда.

Зачем это нужно

Проблема. Если подбирать гиперпараметры и оценивать качество на одних и тех же данных, оценка получается **оптимистично смещённой** – мы как будто подсмотрели ответ. Это частный случай подгонки (слайд 39).

Решение. Nested validation разделяет два вопроса: «какие гиперпараметры лучше?» (внутренний цикл) и «насколько хороша вся процедура в целом?» (внешний цикл). Их данные не пересекаются.

Когда нужно. Когда данных мало и одновременно идёт активный подбор гиперпараметров – то есть почти всегда в исследовательских ИБ-задачах.

Граница темы. Здесь nested validation – только идея «двух непересекающихся вопросов». Математическая детализация и формальные доказательства несмещённости в этом курсе не рассматриваются.



Мини-резюме блока 3.2: какую схему когда применять

Схема	Когда применять	Пример из ИБ
Holdout	Данных много, объекты независимы, нужна одна быстрая оценка	Крупный сбалансированный набор признаков без временной структуры
K-fold CV	Данных мало, нужна устойчивая оценка со средним и разбросом	Небольшой размеченный датасет образцов вредоносного ПО
Stratified	Классы несбалансированы – нужно сохранить пропорцию	95% benign / 5% атак – почти любая задача детектирования
Time-based	Есть время: модель предсказывает будущее по прошлому	Сетевые логи, потоковые события, CIC-IDS-2018 по дням
Group	Есть группы связанных объектов – их нельзя делить	Семейства файлов, пользователи, кампании, IP-источники

Главное правило блока. Выбор схемы определяется **природой данных**, а не удобством. Сначала спросите: есть ли время? есть ли группы? сбалансированы ли классы? – и только потом выбирайте split. Схемы комбинируются: бывает stratified group K-fold или time-based split с группами.



ЧАСТЬ 3 · БЛОК 3 · СЕРДЦЕВИНА ЛЕКЦИИ

Шесть типов утечек данных

Концептуальная типология, единая для всего курса

В этом блоке (слайды 23-32):

- 24 Что такое data leakage – определение
- 25 Тип 1 – через признаки
- 26 Тип 2 – через целевую переменную
- 27 Тип 3 – через preprocessing
- 28 Тип 4 – через дубликаты
- 29 Тип 5 – через время
- 30 Тип 6 – через группу
- 31 Сводная карта всех типов
- 32 Мини-резюме блока



Термин-конвенция: вводится здесь и используется во всех последующих лекциях

“ **Data leakage (утечка данных)** – это попадание в обучение модели информации, **которая недоступна модели в момент реального прогноза в эксплуатации**. В результате качество на тесте оказывается завышенным и не воспроизводится в проде.

Лакмусовая бумажка: один вопрос к каждому признаку

«Будет ли значение этого признака известно в тот момент, когда модель должна сделать прогноз в реальной системе?»

✓ **ДА** – признак допустим

Информация реально доступна на момент прогноза.
Признак можно использовать.

✗ **НЕТ** – это утечка

Признак знает то, чего в проде ещё нет. Использование = leakage, даже если метрика растёт.



Тип 1. Через признаки

25 / 54

Часть 3, блок 3 · Утечки данных

Источник утечки № 1 из 6 в типологии этой лекции



из 6

МЕХАНИЗМ

Среди признаков есть такой, который прямо или косвенно содержит ответ – то есть сам по себе сильно коррелирует с целевой переменной по причине, не относящейся к настоящему предсказанию.

ПРИМЕР ИЗ ИБ

В датасете для предсказания вредоносности файла есть признак `was_blocked_by_firewall`. Но фаервол блокирует файл уже после того, как его признали вредоносным. Модель «угадывает» метку почти идеально – потому что признак фактически и есть метка, только под другим именем.

КАК ПОЙМАТЬ

Подозрительно высокая важность одного признака; near-perfect качество на простой модели; признак, который по смыслу появляется одновременно с меткой или после неё. → *выявление leakage через интерпретацию модели – в L09.*



Тип 2. Целевая переменная

26 / 54

Часть 3, блок 3 · Утечки данных

Источник утечки № 2 из 6 в типологии этой лекции



из 6

МЕХАНИЗМ

Целевая переменная просачивается в признаки через производные величины: агрегаты, которые посчитаны с использованием самой метки, или признаки, сформированные уже после того, как стал известен исход.

ПРИМЕР ИЗ ИБ

Признак `incident_severity_score` рассчитывается SOC-аналитиком уже после того, как событие классифицировано как инцидент. Если подать его в модель предсказания инцидентов, модель «видит» оценку, которой в момент прогноза ещё не существует – это значение целевой переменной в скрытой форме.

КАК ПОЙМАТЬ

Спросите по каждому агрегату: когда он вычислен? Любой признак, в расчёте которого участвует метка (даже частично, даже усреднённо по группе), – кандидат на утечку.



Тип 3. Через preprocessing

27 / 54

Часть 3, блок 3 · Утечки данных

Источник утечки № 3 из 6 в типологии этой лекции



из 6

МЕХАНИЗМ

Масштабирование, кодирование категорий, заполнение пропусков или отбор признаков, обученные на всём датасете до разбиения, передают в train статистику, рассчитанную с участием test. Параметры преобразования «видели» тестовые данные.

ПРИМЕР ИЗ ИБ

Перед split на всём датасете посчитали среднее и дисперсию для StandardScaler. Эти статистики уже включают тестовые объекты – значит, при обучении модель косвенно опирается на распределение test. То же самое с отбором признаков «по всему датасету» и со словарём категорий.

КАК ПОЙМАТЬ

Правило-индикатор: любое преобразование, у которого есть «обучаемые» параметры (среднее, словарь, список отобранных признаков), должно вычисляться только на train. Концепция разбирается на слайдах 33-36. → [техническая реализация защиты \(Pipeline\)](#) – в L10.



Тип 4. Через дубликаты

28 / 54

Часть 3, блок 3 · Утечки данных

Источник утечки № 4 из 6 в типологии этой лекции



из 6

МЕХАНИЗМ

Один и тот же объект – или его почти-копия – попадает и в train, и в test. Модель видит на тесте то, что уже запомнила на обучении, и качество завышается, хотя обобщения не произошло.

ПРИМЕР ИЗ ИБ

В датасете фишинговых писем 90% образцов – варианты одного шаблона кампании: меняются имя адресата и ссылка, остальной текст идентичен. При случайном split варианты одного письма расходятся по train и test. Модель «узнаёт» шаблон, а не учится распознавать фишинг.

КАК ПОЙМАТЬ

Дедупликация перед split: точные дубликаты по хешу и near-duplicates по сходству (например, по расстоянию между текстами или эмбедингами). Проверять пересечение train/test по идентификаторам объектов. → *near-duplicate изображений между train/test – частный случай в L14.*



Источник утечки № 5 из 6 в типологии этой лекции



из 6

МЕХАНИЗМ

Информация из будущего попадает в обучение. Это случается при случайном перемешивании временных данных или когда признак агрегирует значения за период, который в момент прогноза ещё не наступил.

ПРИМЕР ИЗ ИБ

В датасете сетевых событий случайный split раскидал события по train и test без учёта времени. В train попали события, произошедшие позже тестовых. Модель «обучилась на будущем» – а в реальной системе будущего на момент прогноза ещё нет.

КАК ПОЙМАТЬ

Всегда спрашивайте: есть ли в данных временная ось? Если да – только time-based split (слайд 19). Проверяйте, что максимальная дата train строго меньше минимальной даты test. → *детальный разбор time-based split для логов – в L12.*



Тип 6. Через группу

30 / 54

Часть 3, блок 3 · Утечки данных

Источник утечки № 6 из 6 в типологии этой лекции



из 6

МЕХАНИЗМ

Объекты, связанные общей группой (пользователь, устройство, семейство, кампания), разделены между train и test. Похожие объекты группы оказываются по обе стороны – и модель снова «узнаёт», а не обобщает.

ПРИМЕР ИЗ ИБ

События одного пользователя попали и в train, и в test. Модель выучила индивидуальные привычки этого пользователя, а не общие признаки атаки. На новых пользователях в проде такого качества не будет. То же – с файлами одного семейства (кейс со слайда 7).

КАК ПОЙМАТЬ

Определите, что в ваших данных является «группой», и используйте group split (слайд 20). Проверяйте, что идентификаторы групп не пересекаются между train и test. → [group split для аномалий – в L11](#).



Сводная карта шести типов

31 / 54

Часть 3, блок 3 · Утечки данных

Артефакт для повторения: тип → суть → чем закрывается

№	Тип утечки	Суть в одной фразе	Чем закрывается
1	Через признаки	Признак прямо или косвенно содержит ответ	Аудит признаков, интерпретация (L09)
2	Целевая переменная	Метка просочилась через производные признаки	Проверка момента расчёта каждого агрегата
3	Через preprocessing	Преобразование обучено на всём датасете	«Split first» (сл. 33-36), Pipeline (L10)
4	Через дубликаты	Объект или почти-копия в train и в test	Дедупликация перед split
5	Через время	Будущее попало в обучение	Time-based split (сл. 19, L12)
6	Через группу	Связанные объекты в train и в test	Group split (сл. 20, L11)

У всех шести – один корень. Модель получила информацию, которой не будет на момент прогноза в проде. Шесть типов – это шесть мест, где эта информация просачивается. Лакмусовая бумажка со слайда 24 работает для всех.



Мини-резюме блока 3.3: типология data leakage

Главные тейкэвеи

1

Leakage – это информация из «будущего».

Любые данные, недоступные модели на момент реального прогноза.
Один корень – шесть проявлений.

2

Шесть типов – единый чек-лист.

Признаки, целевая, preprocessing, дубликаты, время, группа.
Проверяйте все шесть на каждом проекте.

3

Лакмусовая бумажка универсальна.

«Будет ли это известно в момент прогноза в проде?» Нет → утечка,
даже если метрика растёт.

4

Утечка опаснее всего в ИБ.

Зависимость по времени, источнику, кампании – норма. Случайный split
почти всегда что-то ломает.

Чек-лист перед split

- ☐ Каждый признак: будет ли известен в момент прогноза?
- ☐ Все агрегаты: когда вычислены – до или после метки?
- ☐ Preprocessing: обучается только на train?
- ☐ Дубликаты и near-duplicates удалены до split?
- ☐ Есть временная ось? → time-based split.
- ☐ Есть группы объектов? → group split.



Ч А С Т Ь 3 · Б Л О К 4

Концептуальный принцип защиты

**Сначала разделить – потом
обрабатывать**
split first, transform later

Один принцип закрывает большую часть утечек через preprocessing (тип 3). В этом блоке: определение (сл. 34), частая ошибка (сл. 35), что именно учится только на train (сл. 36).



Любое обучаемое преобразование настраивается только на train

ОПРЕДЕЛЕНИЕ

Сначала данные разбиваются на **train / validation / test**. Затем любое преобразование с обучаемыми параметрами (среднее, дисперсия, словарь категорий, список отобранных признаков) **настраивается только на train** и в неизменном виде применяется к validation и test.

Правильный порядок



Почему это работает. Статистики (среднее, дисперсия, словарь, отобранные признаки) – это уже информация о данных. Если они посчитаны с участием test, test перестаёт быть независимым. Принцип гарантирует: на момент обучения модель «не видела» ни одного тестового объекта даже косвенно.



Самое незаметное нарушение принципа «split first»

⚠ ЧАСТАЯ ОШИБКА

«Я отмасштабирую весь датасет одним `StandardScaler`, а потом разобью на `train` и `test`. Так удобнее – не нужно дважды возиться с преобразованием. **Это же просто нормировка, что тут может утечь?»**

✗ ПОЧЕМУ ЭТО НЕВЕРНО

Среднее и дисперсия скейлера посчитаны по всем объектам – включая `test`. Эти параметры «видели» распределение тестовых данных, и через нормировку эта информация просочилась в `train`. `Test` перестал быть независимым – это утечка через `preprocessing` (тип 3).

Почему так трудно заметить. Эффект маленький и не вызывает ошибок в коде – всё «работает». На умеренных датасетах сдвиг метрики невелик, но на малых данных или сильных выбросах в `test` он может быть значительным. А главное – это методологически неверно всегда.

→ техническая защита: единый `Pipeline`, который сам обучается только на `train` – в `L10`. Здесь – только принцип.



Что учится только на train

36 / 54

Часть 3, блок 4 · Split first

Мини-резюме блока 3.4: один принцип закрывает шесть рисков preprocessing

Эти преобразования всегда настраиваются на train и применяются к val/test:

Масштабирование

StandardScaler, MinMaxScaler – среднее, дисперсия, мин/макс считаются на train

Кодирование категорий

OneHot, target encoding – словарь категорий строится по train

Заполнение пропусков

Imputation – медиана/среднее/мода берутся из train

Отбор признаков

Список отобранных признаков фиксируется по train

Снижение размерности

PCA и подобные – компоненты обучаются на train

Балансировка классов

Oversampling/undersampling – только на train, не трогая test

Запомнить как правило. Если у преобразования есть что «запоминать» из данных – оно должно запоминать только train. Технически это удобно собирать в единый Pipeline (L10), но принцип не зависит от инструмента.



Ч А С Т Ъ 3 · Б Л О К 5

Как обращаться с test set

Одно из самых тонких мест эксперимента – третий источник иллюзии качества (слайд 8)

В этом блоке (слайды 37-40):

- 38 Test – финальная независимая проверка
- 39 Ошибка: «выбрал лучшую из 30 по test»
- 40 Правила работы с test set



На неё смотрят один раз, в самом конце, и после этого модель не меняют

Главный принцип

Test существует, чтобы один раз ответить на вопрос: «какое качество у финальной модели на данных, которых не касалась никакая настройка?»

Как только вы посмотрели на test и приняли любое решение о модели – выбрали порог, поменяли признак, остановили обучение – **test больше не независим**.

Почему «один раз»

Каждый «взгляд» на test с последующим улучшением – это маленькая подгонка под него. По отдельности – почти незаметно. Накопительно – метрика на test всё хуже отражает реальное качество.

Это статистическая, но настоящая утечка: вы оптимизируете решения под конкретную тестовую выборку, а не под реальный мир.

✓ ПРАВИЛЬНЫЙ ПОРЯДОК

Всю настройку – выбор модели, гиперпараметров, признаков, порога – ведём на **validation (или cross-validation)**. Когда модель окончательно зафиксирована – **один раз** прогоняем её на test и записываем результат как финальную оценку. Если после этого что-то меняем в модели – нужен новый, ещё не использованный test.



Ошибка: лучшая модель по test

39 / 54

Часть 3, блок 5 · Работа с test

Многократное сравнение моделей по тесту – это подгонка под шум

⚠ ЧАСТАЯ ОШИБКА

«Я обучил 30 моделей с разными гиперпараметрами, прогнал каждую на test и выбрал ту, у которой метрика на test самая высокая. Это честная оценка – я же измерил на тесте».

✗ ПОЧЕМУ ЭТО НЕВЕРНО

Выбор лучшей по test = настройка по test. Из 30 моделей у части метрика на test высокая не потому, что модель лучше, а потому, что ей «повезло» с конкретной тестовой выборкой. Выбирая максимум из 30, вы выбираете и удачу, и качество вместе.

Итог. Метрика выбранной модели на этом test систематически завышена – в проде она окажется ниже.

✓ **Как правильно.** Сравнивать 30 моделей по validation (или CV), выбрать лучшую, и только её один раз прогнать на test. → для честной оценки с подбором – *nested validation (слайд 21)*.



Мини-резюме блока 3.5: четыре правила и проверка перед публикацией результата

1 Смотрим один раз

Test прогоняется в самом конце, после фиксации модели. Результат записывается как финальная оценка.

2 Не подбираем по нему

Ни гиперпараметры, ни порог, ни признаки, ни early stopping – ничего не настраивается по test.

3 Не сравниваем по нему

Выбор лучшей из нескольких моделей идёт по validation, а не по test.

4 Фиксируем факт взгляда

Если на test всё же посмотрели – это записывается в журнал; для нового решения нужен новый test.

Мнемоника

Test – как запечатанный конверт.

Вся работа идёт на validation. Конверт вскрывают один раз – в самом конце, чтобы объявить результат.

Вскрыли и продолжили править модель? Конверт испорчен – нужен новый.

Этим закрыт третий источник иллюзии качества (слайд 8). Неправильное разбиение и утечки мы уже разобрали – подгонка под test была последней из трёх.



Что нужно для повтора

Шесть вещей, без которых ML-эксперимент нельзя воспроизвести

Версия данных

Какой именно датасет и какая его версия / срез использованы

Код

Исходный код обучения и оценки, зафиксированный (например, в системе контроля версий)

Seed

Зерно генератора случайных чисел – для split, инициализации, перемешивания

Гиперпараметры

Все параметры модели и обучения, а не только итоговые «лучшие»

Окружение

Версии библиотек и среды: одна и та же модель в разных версиях даёт разный результат

Протокол запуска

Схема split, порядок шагов, как именно получены числа в отчёте

⚠ ВАЖНОЕ РАЗЛИЧИЕ

Воспроизводимость кода ≠ воспроизводимость вывода. Код может запускаться без ошибок, но давать другие числа – из-за незафиксированного seed, другой версии библиотеки или изменившихся данных. Цель – чтобы повторный запуск дал **те же метрики**, а не просто «отработал».



Что фиксировать на каждый запуск – обязательный артефакт с первого ДЗ

Что фиксируем	Зачем	Пример записи
Версия датасета	Чтобы знать, на каких данных получен результат	CIC-IDS-2018, дни 1-6 (train), 9-10 (test)
Схема split	Главный фактор честности оценки	time-based, group по src_ip
Параметры модели	Чтобы повторить именно эту конфигурацию	n_estimators=300, max_depth=8, seed=42
Метрики	С указанием выборки (train/val/test)	F1(val)=0.86, F1(test)=0.83
Дата и версия кода	Чтобы связать результат с состоянием кода	2025-09-14, commit a3f9c1

Заведите журнал с первого ДЗ. Это может быть простая таблица или markdown-файл. Без журнала результат «F1 = 0.83» через две недели невозможно ни повторить, ни объяснить. С журналом эксперимент становится знанием, а не разовым эпизодом (вспомните ключевой тезис со слайда 9).



Карточка-напоминание перед разбором практических кейсов

Что это за датасет

CIC-IDS-2018 – публичный датасет сетевых вторжений от Canadian Institute for Cybersecurity.

- ~16 млн записей сетевых потоков (flow)
- ~80 признаков на поток (длительность, байты, флаги)
- Собран за 10 дней
- Benign + 14 типов атак (DDoS, brute force, infiltration, ботнеты и др.)

Почему он – хороший пример

В нём одновременно живут сразу несколько ловушек валидации из этой лекции:

- временная структура – данные по дням → риск утечки через время;
- группы по источникам – одни и те же IP → риск утечки через группу;
- сильный дисбаланс – benign >> атаки → нужен stratified.



Случайное перемешивание CIC-IDS-2018 ломает сразу две вещи

ИБ-КЕЙС · СЕТЕВЫЕ ВТОРЖЕНИЯ

Детектор атак, обученный на случайном split

Постановка. Команда берёт CIC-IDS-2018, перемешивает все ~16 млн потоков и режет случайно 80/20. Обучает классификатор benign/атака. На test – отличные метрики, recall по атакам ≈ 0.95 .

В эксплуатации. На новом трафике (новые дни, новые источники) recall падает до ≈ 0.40 . Модель ловит уже виденные атаки и пропускает новые.

✗ ДИАГНОЗ

Случайный split нарушил сразу два правила. (1) Время: потоки одной атаки растянуты во времени; перемешивание положило начало атаки в train, а продолжение – в test (утечка через время). (2) Группа: один и тот же IP-источник попал и в train, и в test – модель выучила конкретные источники, а не паттерн атаки (утечка через группу).

✓ **Как правильно.** Time-based split (обучение на ранних днях, тест на поздних) **плюс** group split по источнику. Метрика на честном split ниже, зато отражает реальную эксплуатацию.



Кейс 2: фишинг-дубликаты

45 / 54

Часть 4 · Практика ИБ

Когда 80% писем – варианты одного шаблона кампании

ИБ-КЕЙС · ФИШИНГОВЫЕ ПИСЬМА

Классификатор фишинга на датасете из реальных кампаний

Постановка. Датасет собран из перехваченных писем. 80% фишинговых образцов – варианты нескольких массовых кампаний: один шаблон, в котором меняются имя адресата и ссылка. Случайный split 80/20.

В эксплуатации. На отчёте precision/recall высокие. На письмах новых кампаний, которых не было в обучении, модель почти слепа.

Х ДИАГНОЗ

Утечка через дубликаты. Варианты одного шаблона при случайном split попали и в train, и в test. Модель «узнаёт» конкретные шаблоны кампаний, а не учится отличать фишинг от легитимной почты. На test это выглядит как высокое качество, потому что те же шаблоны там и тут.

✓ **Как правильно.** Дедупликация и group split по кампании: все письма одной кампании – в одну выборку. Тест на письмах кампаний, которых модель не видела.



Возврат к мотивационному кейсу со слайда 7 – теперь с полным языком

ИБ-КЕЙС · ВРЕДОНОСНОЕ ПО

Тот самый классификатор PE-файлов (слайд 7)

Напоминание. В начале лекции мы видели: классификатор семейств вредоносного ПО показал $F1 = 0.97$ на test и упал до ≈ 0.60 в проде. Тогда у нас ещё не было языка, чтобы объяснить почему. Теперь есть.

Что именно произошло. Полиморфные варианты одного семейства похожи между собой сильнее, чем разные семейства. Случайный split развёл «братьев-близнецов» по train и test.

Х ДИАГНОЗ

Утечка через группу (семейство). Группа здесь – семейство вредоносного ПО. При случайном split варианты одного семейства оказались в обеих выборках, и модель «узнавала» уже виденные семейства, а не училась различать вредоносное и доброкачественное ПО по сути.

✓ **Как правильно.** Group split по семейству: целые семейства – либо в train, либо в test. Только так оценка показывает способность ловить новые, ещё не виденные семейства.



Что характерно для ИБ-данных

Рамка, через которую смотрим на любой датасет в безопасности

Пять особенностей, из-за которых случайный split в ИБ почти всегда опасен:

Зависимость по времени

События идут потоком; будущее не должно попадать в обучение

Зависимость по источнику

Один IP, пользователь, устройство порождают похожие объекты

Зависимость по кампании

Массовые атаки и рассылки дают много почти-дубликатов

Сильный дисбаланс классов

Атак на порядки меньше, чем нормального трафика

Артефакты сбора данных

Модель может выучить особенности сбора, а не признаки атаки

Дрейф во времени

Атаки эволюционируют; вчерашняя выборка ≠ завтрашний трафик

Вывод. Перед любым ИБ-проектом задайте три вопроса: есть ли время? есть ли группы? сбалансированы ли классы? Ответы определяют схему split раньше, чем вы выберете модель.



Ошибка: «датасет публичный»

Известность датасета не гарантирует корректность вашей валидации

⚠ ЧАСТАЯ ОШИБКА

«Это известный публичный датасет из статей и бенчмарков. Раз все на нём работают, с ним всё в порядке — можно просто случайно разбить и обучаться».

✗ ПОЧЕМУ ЭТО НЕВЕРНО

«Публичный» относится к данным, а не к вашему протоколу. Сам датасет может быть качественным, но способ, которым вы его разбиваете, — целиком на вашей ответственности. CIC-IDS-2018, NSL-KDD и другие классические наборы имеют известную временную и групповую структуру; случайный split на них даёт завышенные метрики независимо от популярности.

Более того, **у самих публичных датасетов есть известные ограничения** — артефакты сбора, дубликаты, устаревшие типы атак. Их нужно знать, а не принимать на веру.

→ критический разбор публичных ИБ-датасетов и их ограничений — в L11.



Если запомнить только семь вещей из L05 – то эти

1**Модель без протокола – число без смысла.**

Качество определяется не только моделью, но и схемой эксперимента.

2**Train / val / test – три разные роли.**

Обучение, выбор, финальная проверка. Подмена ломает оценку.

3**Случайный split – не универсален.**

Есть время → time-based; есть группы → group; дисбаланс → stratified.

4**Шесть типов утечек – единый язык.**

Признаки, целевая, preprocessing, дубликаты, время, группа.

5**Сначала разделить, потом обрабатывать.**

Любое обучаемое преобразование настраивается только на train.

6**Test смотрят один раз.**

Подбор и сравнение – по validation. Test – финальный конверт.

7**Воспроизводимость = журнал.**

Версия данных, split, seed, параметры, метрики, дата запуска.



Область применения принципов лекции и что осталось за её рамками

✓ Применять всегда

- Везде, где есть оценка качества модели – от учебного ДЗ до боевого детектора.
- На любом этапе: выбор признаков, подбор гиперпараметров, сравнение моделей.
- Независимо от алгоритма – принципы валидации не зависят от того, линейная это модель, дерево или нейросеть.

Осталось за рамками L05

- Техническая реализация защиты от утечек – **Pipeline (L10)**.
- Production monitoring и дрейф в эксплуатации – **отдельная тема (L17, L20)**.
- Формальные доказательства несмещённости, bootstrap-интервалы, статтесты сравнения моделей.

Не путать с production monitoring. Валидация отвечает на вопрос «насколько модель хороша до выкатки». Мониторинг – «как она ведёт себя в проде со временем».



L05 – стартовый узел сквозной линии курса

Темы этой лекции вернутся и углубятся в следующих:





Минимум для глубокого понимания темы лекции

УЧЕБНИК

Hastie, Tibshirani, Friedman – The Elements of Statistical Learning

Глава 7 «Model Assessment and Selection»: train/validation/test, cross-validation, оптимизм оценки качества. Базовый текст по теме валидации.

УЧЕБНИК

Bishop – Pattern Recognition and Machine Learning

Раздел 1.3 «Model Selection»: зачем нужна валидация, роль отложенной выборки, идея cross-validation на концептуальном уровне.

СТАТЬЯ

Kaufman, Rosset, Perlich – Leakage in Data Mining (2012)

Классическая работа, вводящая формальную типологию утечек данных и примеры из реальных соревнований. Первоисточник понятия data leakage.



Для тех, кто хочет глубже – особенно в ИБ-контексте

MUST-READ ДЛЯ ИБ-ML

Arp et al. – Dos and Don'ts of Machine Learning in Computer Security (USENIX Security 2022)

Систематический разбор типовых ошибок ML в безопасности, включая утечки и некорректную валидацию. Прямо описывает ловушки из этой лекции на реальных ИБ-задачах.

Практические материалы

- **Документация scikit-learn, раздел Cross-validation:** обзор схем разбиения (KFold, StratifiedKFold, GroupKFold, TimeSeriesSplit) – как справочник, без кода в самой лекции.
- **Документация scikit-learn, раздел про data leakage:** практические рекомендации по предотвращению утечек.
- **Kaggle-разборы реальных случаев leakage** в соревнованиях – для интуиции на конкретных примерах.



Ключевые термины

Train set – выборка для обучения параметров модели

Validation set – выборка для выбора модели и гиперпараметров

Test set – финальная независимая выборка, смотрят один раз

Holdout – однократное разбиение на train/val/test

K-fold CV – K разбиений, каждая часть по очереди – validation

Stratified split – разбиение с сохранением пропорции классов

Time-based split – прошлое в train, будущее в test, без перемешивания

Group split – связанные объекты целиком в одной выборке

Nested validation – вложенные циклы для оценки и подбора отдельно

Data leakage – попадание в обучение недоступной в проде информации

Target leakage – утечка целевой переменной через признаки

Preprocessing leakage – преобразование, обученное на всём датасете

Экспериментальный журнал – запись параметров запуска для воспроизводимости

Воспроизводимость – повторный запуск даёт те же метрики